AFINNI and ART: adaptive surrogates on one refinement engine

A technical companion to the code: the two algorithms, the engine that runs them, and the contracts they keep

PADAS group
The University of Texas at Austin

November 2026

Abstract. A surrogate replaces an expensive function by a cheap one that agrees with it to a stated tolerance. This code builds surrogates out of many small affine models, placed adaptively: where one model is not accurate enough, the region is cut in two and each half gets its own. It implements two ways of doing that. AFINNI [1] clusters a given sample, cuts clusters along the function's curvature and answers queries by a nearest-neighbour search among trust regions. ART [2] cuts the domain itself into polytopes, samples each one afresh and answers queries by walking a tree. Both run on one refinement engine, on CPUs or GPUs and across MPI ranks. This companion follows one two-dimensional example through both algorithms and through the engine. Every idea is shown as a picture, explained in words and stated formally; the pictures are computed from real runs, and the claims about the implementation are the ones its tests check.

Contents

1 An expensive function, a cheap stand-in 1
2 One engine, two plans 3
3 AFINNI: fit, test and what the errors say 5
4 AFINNI: where to cut 8
5 AFINNI: where a leaf can be trusted 11
6 ART: a region for every node 13
7 ART: choosing the plane 16
8 Side by side 20
9 Batches, memory and reproducible sums 21
10 Many ranks 22
11 How we know it is right 24
12 Using it 25
References 25

1 An expensive function, a cheap stand-in

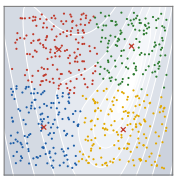
> *A function takes a second to evaluate and is needed a billion times. What can stand in for it, and how much can the stand-in be trusted?*



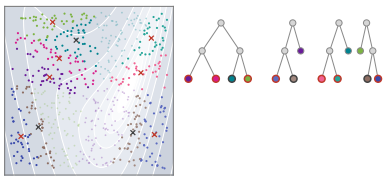
A *surrogate* $\hat{f}$ is a cheap function built from a limited number of evaluations of the expensive f, so that $\hat{f}(x)$ is close to $f(x)$ wherever it will be asked. This companion builds surrogates from the simplest pieces that can work: *affine* models, $\hat{f}(x) = w^\top(x - c) + b$, each valid near its own centre c. A single affine model fits a curved function only over a small region, so the question is where to put the pieces: many where f bends, few where it is nearly flat.

The running example is the function on the left, the curved valley $f(x) = (1 - x_1)^2 + (x_2 - x_1^2)^2$ on the box $[-2, 2] \times [-1, 3]$, drawn by its level sets: light on the valley floor, darker where f is large. The surrogate must agree with it to an absolute tolerance $\alpha_{\text{tol}} = 1$.

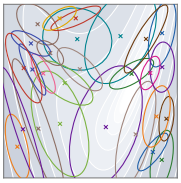
Both algorithms of this code place the pieces *adaptively*. They fit one affine model to a region, measure its error against the tolerance, and where the error is too large they cut the region in two and repeat on each half. They differ in what a region is, where its points come from and how the cut is chosen.



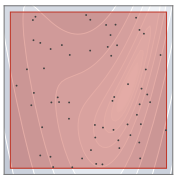
AFINNI starts from a given sample, clustered into a few groups, ...



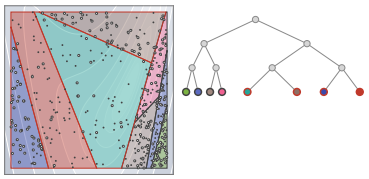
... splits the groups whose model is not accurate enough, ...



... and keeps a trust region around each accepted group.



ART starts from the domain itself, ...



... cuts it by planes, drawing fresh points in each part, ...



... and ends with an exact partition into polytopes.

	AFINNI	**ART**
a region is	a *cluster* of sample points	a *polytope* of the domain
its points	are given; a few are added inside small clusters	are drawn by the node itself, uniformly in its polytope
the cut	follows the curvature of f, estimated from the points	is searched for: the plane whose two fits are best
what is kept	only the leaves, each with a trust ellipsoid	the whole tree, with every plane
a query	finds the nearest leaf in the leaves' own metrics, and may be flagged as outside every trust region	walks down the tree to exactly one leaf

AFINNI suits a sample that already exists, such as the states a simulation visited: it never needs to know the domain. ART suits a known domain on which every point must get an answer. In the code both are *plans* of one *refinement engine* (Section 2), chosen by `TrainingConfig(algorithm="afinni")` or `"art"`.

1.1 How to read this companion

Each idea appears three times: as a **picture** of the running example, in plain **words**, and in **symbols**, as a definition, lemma or theorem in a tinted box. The pictures and the symbols say the same thing, so either can be used to check the other. The pictures are computed, not drawn: `figures/make_figures.py` trains both algorithms on the example, records every level of the runs, and stops unless what it recorded reproduces the trained models exactly. It also writes the numbers this text quotes.

Colours keep one meaning throughout. In a map of clusters each cluster has its own colour, with a × dark cross at its centre if its model was accepted and a × red one if it was rejected. In a map of a model's *errors* at its points, a point is ○ pale where the model is within the tolerance and deep ● blue or ● red where it is too high or too low. Highlighted pseudocode lines are the ones the picture beside them shows.

The notation follows the AFINNI manuscript [1], and ART reuses it where the objects are the same.

In words	In symbols
the function, its surrogate, the input dimension	f, $\hat{f}$, d
the initial points and their number	X_0, N (`X_train`)
a cluster, its points, its centre	C, X_C, c
the local model of a cluster	$\hat{f}(x) = w^\top(x-c)+b$ (`A_fit`, and `b_fit` $= b - w^\top c$)
the absolute and relative tolerances	α_{tol}, r_{tol} (`atol`, `rtol`)
the number of initial clusters; the minimum number of points in a cluster	N_{init}, $N_{\min}$ (`num_initial_clusters`, `min_pts_per_cluster`)
the Hessian of f and its absolute value	H, $\lvert H\rvert = U\lvert D\rvert U^\top$ if $H = UDU^\top$
the trust region of a leaf	$(x-c)^\top T(x-c) \le 1$ (T is `Q` in the code)
the leaves and their number	$\mathcal{S}$, N_c
ART: a node's region, in coordinates $u \in [0,1]^d$ of the domain	$\Omega_C = \{u : Au \le z\}$
ART: a split plane; the side $a^\top u \ge z$ is the child C_1	(a, z), $\lVert a\rVert = 1$ (`normal`, `offset` in the code)
ART: a node's path code, and its number of points	π, $n = \lceil mp\rceil$ with $p = d+1$
the MPI ranks, and the work of rank i	P, w_i

In one line. Fit an affine model, test it against the tolerance, cut where it fails: AFINNI does this to clusters of a given sample, ART to polytopes of a known domain.

2 One engine, two plans

> *Two algorithms that fit, test and cut. What do they share, and what is each one's own?*

Everything except what happens to one batch of clusters. Both algorithms keep a *queue* of clusters waiting to be fitted, process it in batches, send accepted clusters to an output buffer and the children of rejected ones to the next level's queue. The *refinement engine* owns that loop, with the memory management, the transfers to the GPU, the MPI distribution and the work stealing. An algorithm is a *plan*: it says what the initial queue is, where a batch may end, how much memory a point needs, and what to do with one batch.

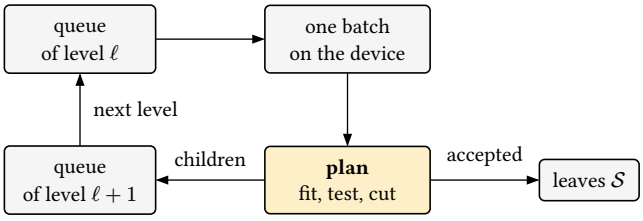
Algorithm 1: The refinement engine: `refine` in `src/engine/driver.py` and `refine_rank` in `src/engine/refinement.py`. Lines 5–6 are the plan's.

Require: a plan (AFINNI or ART), the oracle f, tolerances α_{tol}, r_{tol}, a depth limit
Ensure: the leaves $\mathcal{S}$, each with its local model

```
1 queue ← the plan's initial clusters                    // rank 0, then scattered (Section 10)
2 while this rank has clusters, or can steal some
3     for each batch of whole clusters that fits the device    // Section 9
4         answer steal requests from the tail of the queue     // Algorithm 9
5         leaves, children ← the plan processes the batch      // Algorithm 2 or Algorithm 6
6         append the leaves to S and the children to the next queue
7     queue ← the next queue, without children beyond the depth limit
8 gather S on rank 0                                     // ART also assembles its tree
```

A level is one pass over the queue. Clusters never move between levels except by being split, so in a single process the queue of level ℓ holds exactly the clusters at depth ℓ of the forest. The loop ends when the queue is empty: every cluster has either become a leaf or reached the depth limit.

Processing whole *levels*, instead of following one branch down, is what makes the algorithm parallel: all clusters of a batch are fitted, tested and cut at once, as array operations over thousands of clusters (Section 9).



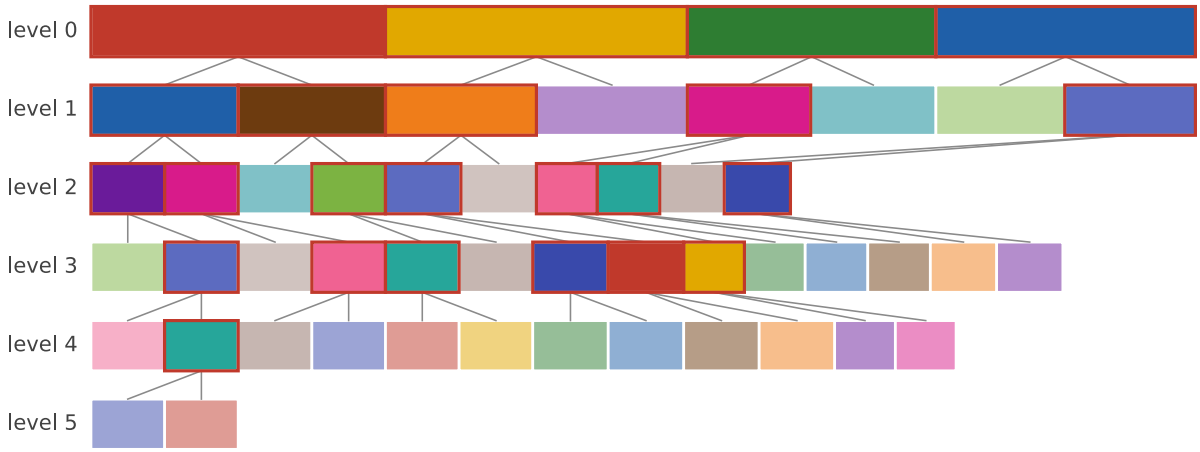


Figure 1: The queue of every AFINNI level of the running example, as it is stored: each cluster's points back to back, in one array. Accepted clusters (pale) leave for the leaves; each rejected cluster (outlined red) is replaced in the next level by its two children, next to each other. Some rows grow: a small rejected cluster is given new points before it is cut (Section 4.2).

In words	In symbols
a cluster is a run of consecutive rows	rows `starts[k]` to `starts[k] + sizes[k]` of `X`, `F`
a batch is a run of consecutive clusters	`plan.batch_end(...)` chooses where it ends
the children of a cluster are adjacent	children $2j$ and $2j+1$ of the next queue belong to the j-th cluster cut at this level

> **Invariant 1** (queue order). Batches are taken in queue order, rejected clusters are cut in queue order, and each appends its two children consecutively. Within one rank the queue of every level is therefore sorted by position in the forest. Nothing has to be sorted or looked up between levels, and a rank can give away the tail of its queue as one contiguous block.

The plans differ in the following, and in nothing else. `tests/unit/test_plans.py` checks that both provide every member of the `Plan` protocol in `src/core/plan.py`.

	AFINNI (`AFINNIPlan`)	**ART (`ARTPlan`)**
initial queue	the N_{init} clusters of the initial sample	the roots: the domain, or a grid or Voronoi cells of it
points of a cluster	its own, plus new ones if it is small and rejected	topped up to n inside its polytope before fitting
extra columns	none	root, path code π, the rows of Ω_C
cut	Hessian-informed, at the median	searched plane (a, z), recorded
at the depth limit	the cluster is dropped and reported	the node becomes a leaf, flagged
after the run	nothing	assembles and checks the tree

In one line. The engine owns the queue, the batches, the memory and the ranks; a plan owns what a cluster is and what happens to one batch.

3 AFINNI: fit, test and what the errors say

> *A cluster of points with known values of f. Is one affine model enough for it?*

AFINNI begins by clustering the initial points X_0 into N_{init} groups with k-means: the roots of a forest of binary trees. Then every level asks the same question of every cluster. The answer is computed, not guessed: fit the best affine model to the cluster's points, and look at how far it is from f at those same points.

Definition 1 (local model and cluster errors). For a cluster C with points X_C and centre c (their mean), the *local model* is $\hat{f}(x) = w^\top(x - c) + b$ with

$$(w, b) = \arg\min_{w,b} \sum_{x_j \in X_C} \left| f(x_j) - w^\top (x_j - c) - b \right|^2. \tag{1}$$

Its *absolute* and *relative errors* are

$$e_a = \max_{x \in X_C} \left| f(x) - \hat{f}(x) \right|, \quad e_r = \frac{e_a}{\max_{x \in X_C} |f(x)|}. \tag{2}$$

The cluster is *accepted* if $e_a < \alpha_{\text{tol}}$ or $e_r < r_{\text{tol}}$, and *rejected* otherwise.

Algorithm 2: AFINNI on one batch of clusters (the manuscript's Algorithm 1, batched): `AFINNIPlan.process_batch` in `src/afinni/plan.py`.

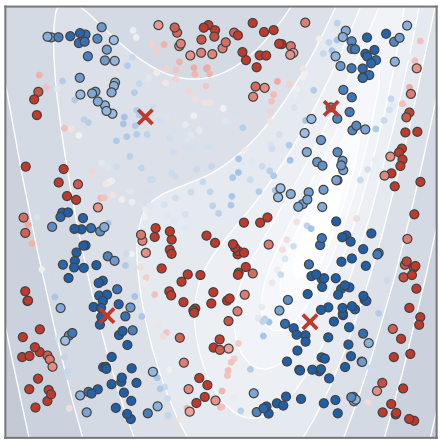
Require: a batch of clusters C with points X_C and values $f(X_C)$; $N_{\min}$, α_{tol}, r_{tol}
Ensure: leaves $(c, (w, b), T)$ and, for the next level, two children per rejected cluster

parallel for each cluster C
  $c \leftarrow$ mean of X_C; evaluate $f(c)$
  $(w, b) \leftarrow \arg\min \sum_{x_j \in X_C} \left| f(x_j) - w^\top (x_j - c) - b \right|^2$ *// Eq. 1*
  $e_a, e_r \leftarrow$ the cluster's errors *// Eq. 2*
parallel for each cluster with $e_r < r_{\text{tol}}$ or $e_a < \alpha_{\text{tol}}$ *// accepted*
  $T \leftarrow$ trust ellipsoid of X_C *// Algorithm 4*
  add $(c, (w, b), T)$ to the leaves
parallel for each other cluster *// rejected*
  if $\#C < 2N_{\min}$: sample new points in C and evaluate f there *// Section 4.2*
  $|H| \leftarrow$ Hessian estimate *// Algorithm 3*
  $C_{-1}, C_1 \leftarrow$ Hessian-informed split *// Algorithm 3*
  append C_{-1} and C_1, next to each other, to the next level

1 Level 0: no cluster passes

Algorithm 2

⋮
Ensure: leaves $(c, (w, b), T)$ and, for the next level, two children per rejected cluster
1 **parallel for** each cluster C
2 $c \leftarrow$ mean of X_C; evaluate $f(c)$
3 $(w, b) \leftarrow \arg\min \sum_{x_j \in X_C} \left| f(x_j) - w^\top (x_j - c) - b \right|^2$
4 $e_a, e_r \leftarrow$ the cluster's errors
5 **parallel for** each cluster with $e_r < r_{\text{tol}}$ or $e_a < \alpha_{\text{tol}}$
⋮

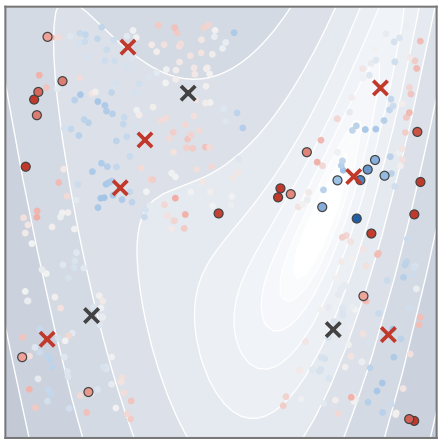


The 4 initial clusters, with every point coloured by the error of its cluster's model. Each cluster shows the same pattern: the model is too low at the rim and too high in the middle, or the reverse. That bowl is the part of f an affine model cannot follow, its curvature. The worst error is 11.59, far above $\alpha_{\text{tol}} = 1$, so all 4 clusters are rejected (red crosses).

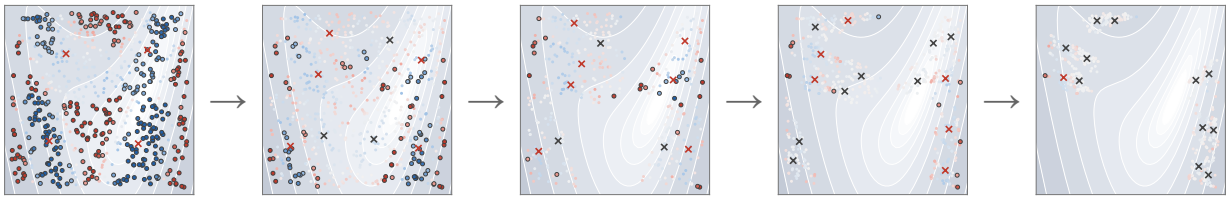
2 The errors shrink with the clusters

Algorithm 2

⋮
4 $e_a, e_r \leftarrow$ the cluster's errors
5 **parallel for** each cluster with $e_r < r_{\text{tol}}$ or $e_a < \alpha_{\text{tol}}$
6 $T \leftarrow$ trust ellipsoid of X_C
⋮



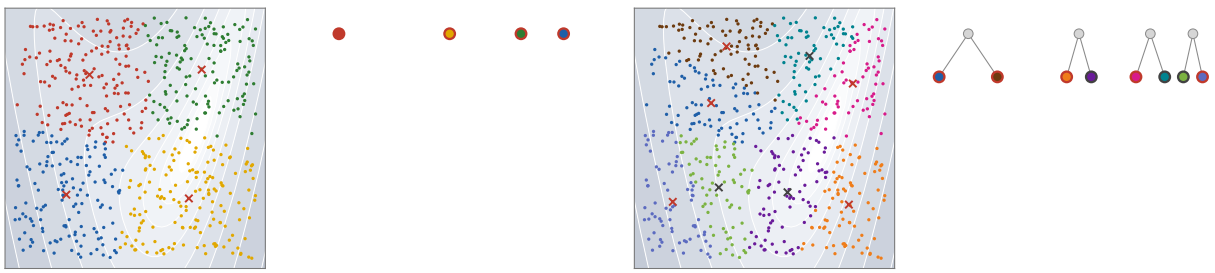
Two levels later there are 10 clusters, a quarter of the size. The error of an affine model falls with the square of a cluster's width, so the colours have faded: 3 clusters now pass (dark crosses) and retire as leaves. The others, where f bends most, are cut again.



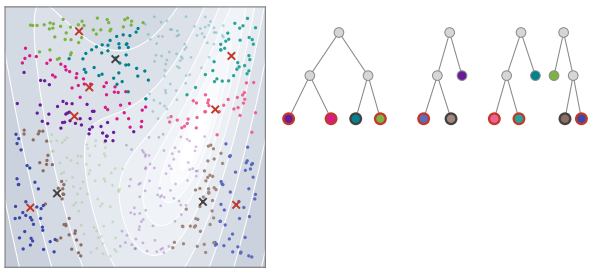
level 0: 11.59 level 1: 6.12 level 2: 4.68 level 3: 2.4 level 4: 1.41

Figure 2: The clusters still in the queue at each level, coloured by the error of their models in units of the tolerance, with the worst error of each level. Accepted clusters leave the picture; the rest are cut and fitted again. After 6 levels nothing is left.

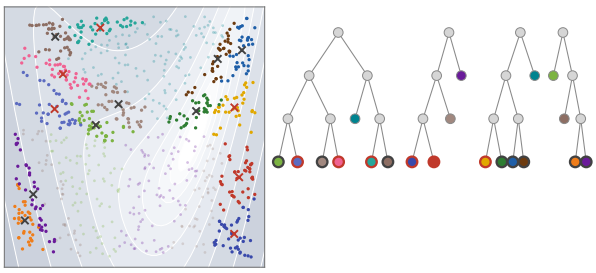
The same run, seen as clusters and as the forest they form:



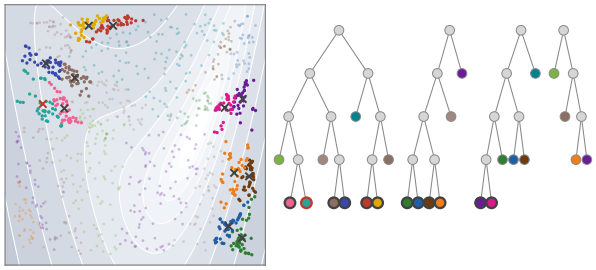
level 0: 4 clusters, 0 accepted level 1: 8 clusters, 3 accepted



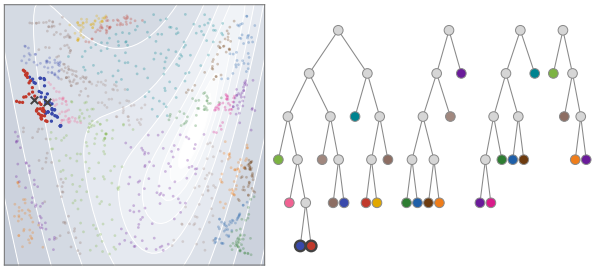
level 2: 10 clusters, 3 accepted



level 3: 14 clusters, 8 accepted



level 4: 12 clusters, 11 accepted



level 5: 2 clusters, 2 accepted

Each frame shows the clusters of one level in colour and the leaves of earlier levels faded, beside the forest down to that level: a node ringed dark was accepted, one ringed red was cut. The run ends with 27 leaves after 1189 evaluations of f: the 600 initial points, 539 new ones, and one at the centre of every cluster.

Lemma 1 (the fit is at least as good as a Taylor expansion). For every cluster,

$$\sum_{x_j \in X_C} \left| f(x_j) - \hat{f}(x_j) \right|^2 \leq \sum_{x_j \in X_C} \left| f(x_j) - f(c) - \nabla f(c)^\top (x_j - c) \right|^2.$$

Proof. The Taylor expansion at c is an affine function, and $\hat{f}$ minimizes the sum of squares over all affine functions. □

So the error of a leaf can be reasoned about as a Taylor remainder, which is $\frac{1}{2}(x-c)^\top H (x-c)$ to leading order: the bowl in the pictures. This is what the next section uses to decide where to cut.

In the code. `AffineLSQModel` (`src/components/local_models.py`) solves Eq. 1 for every cluster of a batch at once, with a small ridge, $10^{-8}\,\mathrm{tr}(\Sigma)$, so that a cluster lying in a subspace still has a solution. Vector-valued f gets one row of w per component, and a cluster passes only if every component passes; tolerances can be given per component. `error_metric="L2"` replaces the maxima in Eq. 2 by root-mean-squares. Other local models (a Taylor model from oracle gradients, a constant, a quadratic) are registered under `SurrogateMode` and `LOCAL_MODELS`.

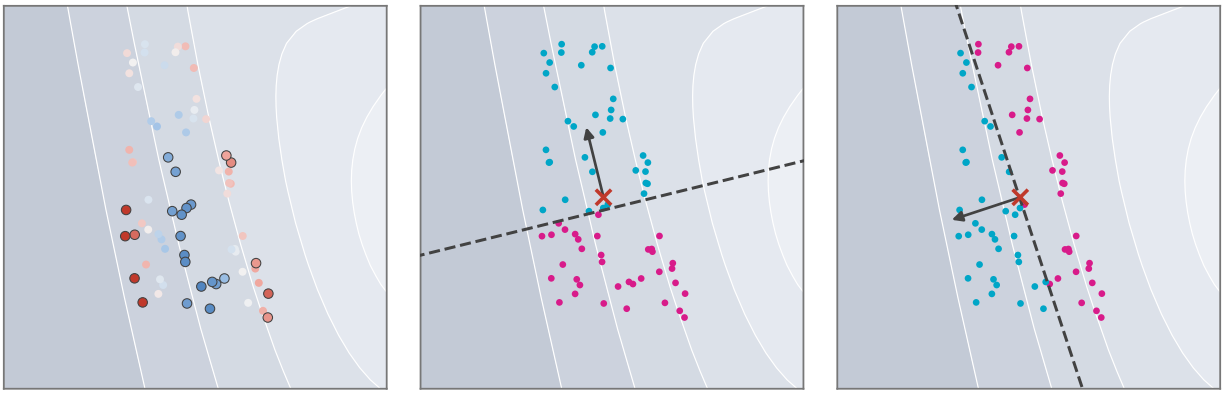
What acceptance does not say. The test is made on the points the model was fitted to, so it is optimistic, and e_r divides by the cluster's largest $|f|$, not by each point's own. An accepted leaf can therefore hold a point whose own relative error is large, where f is near zero. The tolerance is a property of the training points; Section 8 measures what it gives on the whole domain.

In one line. A cluster is accepted when its best affine model is within the tolerance on its own points; the pattern of the errors that remain is the curvature of f.

4 AFINNI: where to cut

> *A cluster is rejected and must be cut in two. Along which direction?*

The obvious answer is the cluster's longest direction: that is 2-means clustering, or a cut across the first principal component [3]. It looks only at where the points are. But a cut is useful only if the two halves are easier to fit than the whole, and that depends on f.



a rejected cluster:
the error of its model

cut across the longest direction:
children's worst error 2.82

Hessian-informed cut:
children's worst error 1.13

Figure 3: One rejected cluster of level 1, with 71 points and worst error 2.87. Its error forms stripes: f bends *across* the cluster and hardly at all along it. A cut across the longest direction leaves both children with the full bend. A cut across the bend, 86 degrees away, leaves children that are nearly flat.

The stripes of the left frame are the level sets of the error, and the error is the Taylor remainder $\frac{1}{2}(x-c)^\top H(x-c)$. So the cut should be made across the direction in which that quadratic form grows fastest. The trick is to measure distances with the curvature: stretch the cluster by $|H|^{1/2}$, so that a direction in which f bends strongly becomes long, and then cut the stretched cluster across *its* longest direction.

Theorem 1 (Hessian-informed two-means, [1]). Let f be a quadratic, possibly non-convex, with Hessian H, and let $\hat{X}$ hold the centred points of a cluster as its columns. For the sum of the two children's errors $E = \sum_{k=\pm 1} \sum_{i \in C_k} \left| f(x_i) - \hat{f}(x_i) \right|$, with $\hat{f}$ the least-squares model of each child, the optimal continuous two-means indicator is the first eigenvector of $\hat{X}^\top |H| \hat{X}$.

The eigenvector of the $\#C \times \#C$ matrix $\hat{X}^\top |H| \hat{X}$ is never formed. The $d \times d$ matrix $\sum_i \tilde{x}_i \tilde{x}_i^\top$ of the stretched points $\tilde{x}_i = |H|^{1/2}(x_i - c)$ has the same nonzero eigenvalues, and projecting the stretched points on its first eigenvector u_1 gives the same indicator.

4.1 Estimating the curvature from values alone

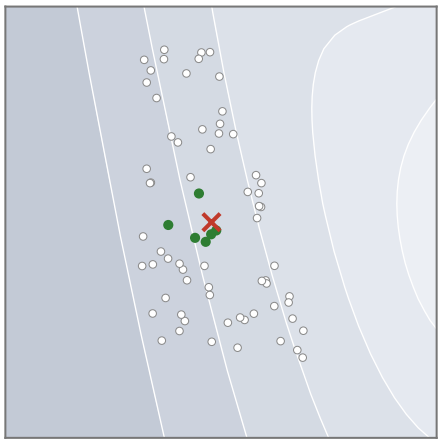
Theorem 1 needs $|H|$, and the oracle gives only values of f. The estimate uses the same pattern the pictures show: the points where the Taylor expansion is nearly exact lie in a small sublevel set of the remainder around c, which is an ellipsoid with the shape of $|H|^{-1}$. Their scatter matrix is therefore proportional to $|H|^{-1}$.

1 A gradient from the nearest points

Algorithm 3

```
⋮
Ensure: children C_{-1}, C_1
ĝ ← least-squares gradient at c from the points nearest to c
e_i ← |f(x_i) − f(c) − ĝ^⊤(x_i − c)| for every point
⋮
```

The Taylor expansion needs $\nabla f(c)$. It is fitted by least squares to the 6 points nearest to c (green), where the expansion is most accurate, using their values and $f(c)$.

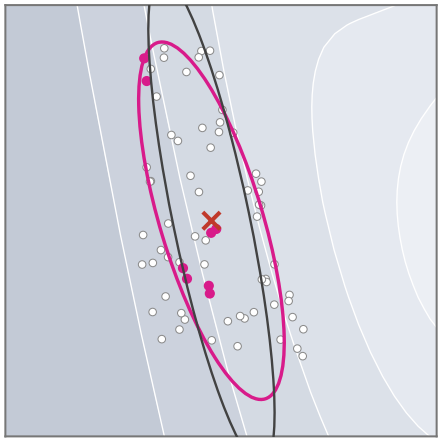


2 The level set of the Taylor error

Algorithm 3

```
⋮
ĝ ← least-squares gradient at c from the points nearest to c
e_i ← |f(x_i) − f(c) − ĝ^⊤(x_i − c)| for every point
I ← the N_cov points with the smallest e_i
Σ ← Σ_{i∈I} (x_i − c)(x_i − c)^⊤;  |H|^{1/2} ← Σ^{-1/2}
x̃_i ← |H|^{1/2}(x_i − c)
⋮
```

The 8 points with the smallest Taylor error (pink) trace the directions in which f is nearly affine. The ellipse of their scatter matrix (pink) has the shape of $|H|^{-1}$: long where f is flat, short where it bends. The dark ellipse is the exact $|H|^{-1}$ at c, scaled to the same area for comparison, which the algorithm never sees.

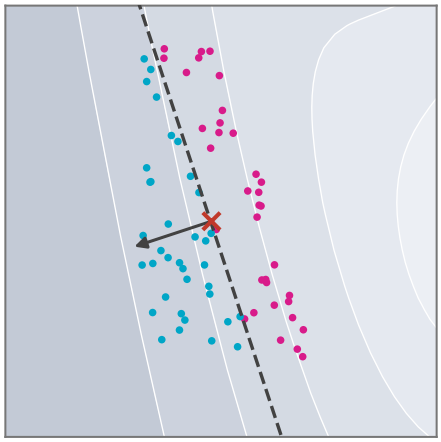


3 Stretch, project, cut at the median

Algorithm 3

```
⋮
Σ ← Σ_{i∈I} (x_i − c)(x_i − c)^⊤;  |H|^{1/2} ← Σ^{-1/2}
x̃_i ← |H|^{1/2}(x_i − c)
u_1 ← first eigenvector of Σ_i x̃_i x̃_i^⊤
s_i ← u_1^⊤ x̃_i; sort the points by s_i
C_{-1} ← the lower half, C_1 ← the upper half
```

In the metric $|H|^{1/2}$ the short axis of that ellipse becomes the long one. The arrow is the resulting direction $|H|^{1/2}u_1$; the points are sorted by their score s_i along it and divided into a lower half C_{-1} (pink) and an upper half C_1 (cyan).



Algorithm 3: Hessian estimate and Hessian-informed split of one rejected cluster (the manuscript's Algorithms 2 and 3): `CovarianceHessianSplit` in `src/components/splitting.py`.

Require: points X_C of a rejected cluster, their values, the centroid c and $f(c)$
Ensure: children C_{-1}, C_1

$\hat{g} \leftarrow$ least-squares gradient at c from the points nearest to c
$e_i \leftarrow \left| f(x_i) - f(c) - \hat{g}^\top (x_i - c) \right|$ for every point *// Taylor error*
$\mathcal{I} \leftarrow$ the N_{cov} points with the smallest e_i
$\Sigma \leftarrow \sum_{i \in \mathcal{I}} (x_i - c)(x_i - c)^\top$; $|H|^{1/2} \leftarrow \Sigma^{-1/2}$ *// Theorem 2*
$\tilde{x}_i \leftarrow |H|^{1/2}(x_i - c)$ *// stretch along the curvature*
$u_1 \leftarrow$ first eigenvector of $\sum_i \tilde{x}_i \tilde{x}_i^\top$
$s_i \leftarrow u_1^\top \tilde{x}_i$; sort the points by s_i
$C_{-1} \leftarrow$ the lower half, $C_1 \leftarrow$ the upper half *// median cut*

Theorem 2 (the scatter of a level set estimates the Hessian, [1]). Let f be a convex, non-degenerate quadratic with Hessian H, and let $E_\varepsilon = \left\{ x : \left| f(x) - \hat{f}_c(x) \right| \le \varepsilon \right\}$, with $\hat{f}_c$ the Taylor expansion at c. If $x_1, \ldots, x_n$ are independent and uniform on E_ε and $\hat{\Sigma} = \frac{1}{n} \sum_i (x_i - c)(x_i - c)^\top$, then for some constants K and κ_0, with probability $1 - e^{-v}$,

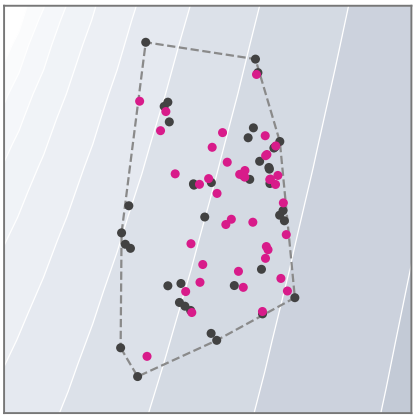
$$\left\| \hat{\Sigma}^{-1} - \kappa_0 H \right\|_2 \lesssim K \|H\| \sqrt{\frac{d+v}{n}}.$$

The factor κ_0 does not matter: only the direction of the first eigenvector is used. On the example the Hessian-informed cut left the children with a smaller worst error than the cut across the longest direction in 21 of the 23 clusters that were split.

In the code.

- The gradient uses the $\max(2d, \lfloor N_{\min}/4 \rfloor)$ nearest points, the level set $N_{\text{cov}} = \lfloor N_{\min}/3 \rfloor$ points.
- The cut is at the *median* of the scores by default, so the two children have equal size and the forest stays balanced. `cut="sign"` cuts at $s_i = 0$ as in the manuscript's Algorithm 2.
- The assumptions of Theorem 2 rarely hold exactly: f is not quadratic, not convex, and the points are not uniform on a level set. In practice the estimate recovers the dominant directions, which is what the cut needs. Where an exact gradient or Hessian is available, the modes `AFFINE_COV_HESS_EXACT_GRAD` and `AFFINE_EXACT_HESS` use it.

4.2 Enough points to estimate anything



A gradient, a Hessian and two children's models cannot be estimated from a handful of points. Before a rejected cluster with fewer than $2N_{\min}$ points is cut, it is given new ones, up to $\max(2\#C, 2N_{\min})$, so that each child starts with at least $N_{\min}$. Here a cluster of 41 points (dark) receives 41 new ones (pink).

A new point is a random convex combination of two of the cluster's points, $\tilde{x} = \lambda x_i + (1 - \lambda) x_j$ with λ uniform in $[0, 1]$, and f is evaluated there. This is one array operation on a GPU and needs no knowledge of the cluster's boundary.

Lemma 2 (new points stay inside). Every new point lies in the convex hull of its cluster's points (dashed in the picture).

The hull is not the cell. Lemma 2 is the method's limit as well as its safety. New points never leave the hull, so the space between the hulls of neighbouring clusters is never sampled, and they concentrate towards the middle of the hull. When many points must be generated from few, the leaves cover less of the domain, and more queries fall outside every trust region (Section 5). The remedy within AFINNI is a larger initial sample. ART samples each region up to its boundary instead (Section 6).

In one line. Cut across the direction in which f bends, not across the direction in which the points spread: stretch the cluster by $|H|^{1/2}$, estimated from the points where the Taylor expansion is nearly exact, and cut the stretched cluster in half.

5 AFINNI: where a leaf can be trusted

A query point arrives. Which leaf should answer it, and should the answer be believed?

AFINNI keeps no tree and no boundaries, only the leaves. Each leaf knows where its model was tested: at its own points. So each leaf carries a region that contains them.

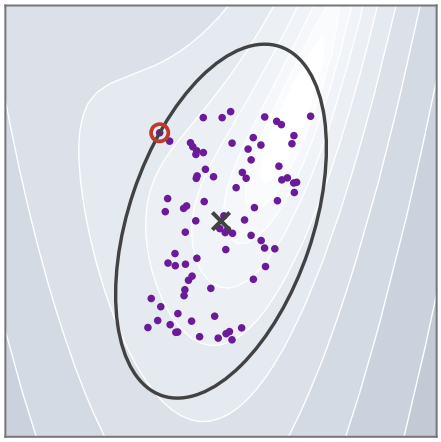
1 An ellipsoid around a leaf's points

Algorithm 4

Require: points X_C of an accepted cluster, centred at c; regularization γ
Ensure: T positive definite, with $(x-c)^\top T(x-c) \le 1$ for every $x \in X_C$

1 $\Sigma \leftarrow \sum_i (x_i - c)(x_i - c)^\top = U \Lambda U^\top$
2 $\Sigma \leftarrow U \max(\Lambda, \gamma) U^\top$
3 $\alpha \leftarrow \max_i (x_i - c)^\top \Sigma^{-1} (x_i - c)$
4 $T \leftarrow \Sigma^{-1} / \alpha$

The scatter matrix Σ of the leaf's 82 points gives the shape: the ellipsoid $\{x : (x-c)^\top \Sigma^{-1}(x-c) \le 1\}$ is long where the points spread. Scaling it by the largest value α that $(x_i - c)^\top \Sigma^{-1}(x_i - c)$ takes puts the farthest point (ringed) exactly on the boundary and every other point inside.



Algorithm 4: The trust ellipsoid of an accepted cluster (the manuscript's Algorithm 5): `CovarianceEllipsoid` in `src/components/trust_regions.py`. γ is `q_reg`.

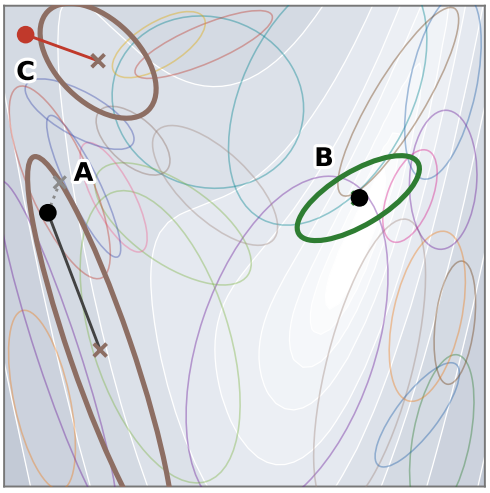
Require: points X_C of an accepted cluster, centred at c; regularization γ
Ensure: T positive definite, with $(x-c)^\top T(x-c) \le 1$ for every $x \in X_C$

1 $\Sigma \leftarrow \sum_i (x_i - c)(x_i - c)^\top = U \Lambda U^\top$
2 $\Sigma \leftarrow U \max(\Lambda, \gamma) U^\top$ *// no flat directions*
3 $\alpha \leftarrow \max_i (x_i - c)^\top \Sigma^{-1} (x_i - c)$ *// the farthest point*
4 $T \leftarrow \Sigma^{-1} / \alpha$

Lemma 3 (the trust region contains the leaf's points). With T from Algorithm 4, $(x_i - c)^\top T (x_i - c) \le 1$ for every point x_i of the cluster, with equality for the farthest one.

Proof. $(x_i - c)^\top T (x_i - c) = (x_i - c)^\top \Sigma^{-1} (x_i - c) / \alpha$, and α is the largest numerator. □

A minimum-volume enclosing ellipsoid [4] would be tighter, but needs an iterative optimization per leaf; this one is a single eigendecomposition, batched over all accepted clusters.



The ellipsoids do two jobs at inference. They are a *metric*: the distance from a query x to leaf i is $d_i = (x - c_i)^\top T_i (x - c_i)$, which counts a step along a leaf's long axis as short. And they are a *flag*: if $d_i > 1$ for every candidate, no leaf has tested its model near x.

A: the nearest centre in Euclidean distance (grey cross) is not the leaf that answers. The long leaf's centre is farther away, but x lies well inside its ellipsoid ($d_\nu = 0.54$).
B: x lies inside 2 ellipsoids; the smallest d_i decides.
C: x lies inside none ($d_\nu = 1.59$). The nearest leaf still answers, and the answer is returned *flagged* (red).

Algorithm 5: Inference (the manuscript's Algorithm 6): `AFINNISurrogate.predict` with `predictor="knn"`.

Require: the leaves $(c_i, (w_i, b_i), T_i)$, a query x, a number of candidates κ
Ensure: $\hat{f}(x)$, and whether x is inside a trust region

$\mathcal{C}_x \leftarrow$ the κ centres nearest to x *// Euclidean, exact*
$d_i \leftarrow (x - c_i)^\top T_i (x - c_i)$ for each $c_i \in \mathcal{C}_x$
$\nu \leftarrow \arg\min_i d_i$
$\hat{f}(x) \leftarrow w_\nu^\top (x - c_\nu) + b_\nu$
return $\hat{f}(x)$, and "inside" if $d_\nu \le 1$, "outside" otherwise

Searching all N_c leaves in their own metrics would cost $O(N_c d^2)$ per query. The search is therefore done in two stages: an exact Euclidean search for κ candidates (`src/neighbors/knn.py`), then the κ trust-region distances. κ should grow with the dimension and with the anisotropy of the leaves; the manuscript uses $\kappa = 2d^2$.

In the code. `predict(X, k_nn=1, coarse_factor=10)` searches $\kappa =$ `k_nn * coarse_factor` candidates and returns the predictions with a mask of the flagged queries. With `k_nn > 1` the best `k_nn` leaves are blended with weights $1/d_i^2$, which smooths the jumps between leaves. On the example, queries A, B and C are answered with 10.65, 0.28 and 7.18, where f is 10.97, 0.13 and 9.74.

A trust region is not a certificate. Inside a trust region the model was tested at the leaf's points, not everywhere. The flag says "no leaf has data near here"; its absence does not bound the error. In high dimension the ellipsoids cover a vanishing share of a box, so the flag is informative only if the sample lies where the queries will. The manuscript's *iterative refinement* uses the flag as a sampling rule: draw new initial points, keep those outside every trust region, and train again.

In one line. A leaf answers for the ellipsoid that contains its points; a query is answered by the leaf nearest in the leaves' own metrics, and flagged if it is inside none.

6 ART: a region for every node

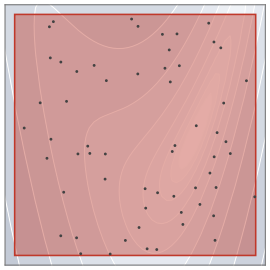
> *AFINNI never knows where a cluster ends. What changes if every node of the tree owns a precise piece of the domain?*

Three things: the node can be sampled up to its boundary, the leaves tile the domain with no gaps, and a query can be sent to its leaf by a few comparisons. This is ART, the adaptive regression tree [2]. It needs what AFINNI does not, a domain, given as a box (`domain=(lower, upper)`); all its geometry is done in the coordinates $u \in [0,1]^d$ of that box.

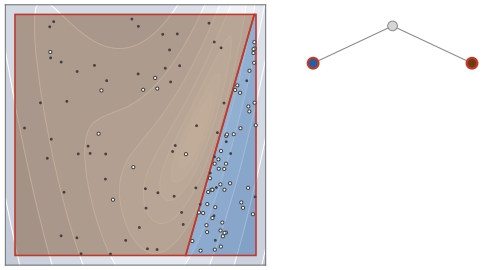
Definition 2 (regions, planes and path codes). The root's *region* is the domain, $\Omega = [0,1]^d$ in u. A node C that is split by the plane (a, z), $\|a\| = 1$, has the children

$$\Omega_{C_{-1}} = \Omega_C \cap \{u : a^\top u \le z\}, \quad \Omega_{C_1} = \Omega_C \cap \{u : a^\top u \ge z\},$$

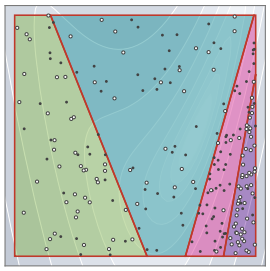
and a point with $a^\top u = z$ belongs to C_1. A region is therefore a convex polytope $\{u : Au \le \boldsymbol{z}\}$: the box, and one row $\pm(a, z)$ of $(A, \boldsymbol{z})$ for every ancestor. The *path code* of the root is $\pi = 1$, and the children of node π are 2π (for C_{-1}) and $2\pi + 1$ (for C_1): one binary digit per level.



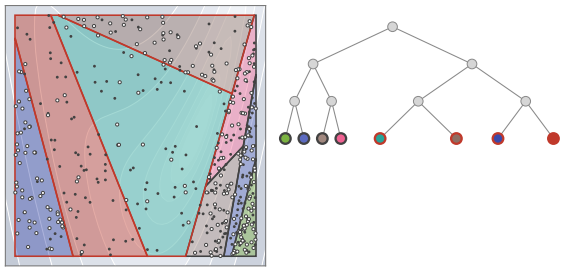
depth 0: the domain, 60 points



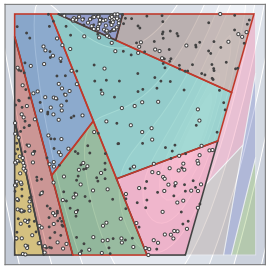
depth 1: 2 nodes, 60 new points



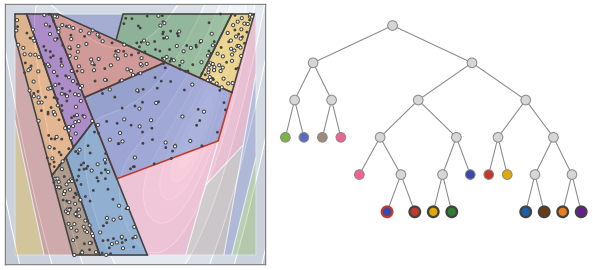
depth 2: 4 nodes



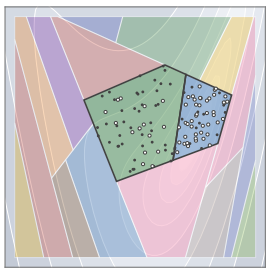
depth 3: 8 nodes, 4 accepted



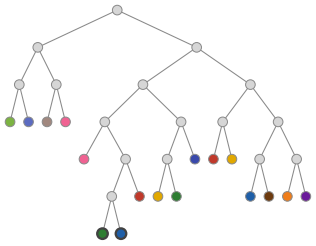
depth 4: 8 nodes, 4 accepted



depth 5: 8 nodes, 7 accepted



depth 6: 2 nodes, 2 accepted





the 17 leaves

Each frame shows the nodes of one depth: a region outlined dark was accepted, one outlined red is cut in the next frame, and leaves of earlier depths are faded. The dots are the node's points: dark ones were inherited from its parent, open ones are new. Compare with AFINNI's frames in Section 3: the pieces now have edges, and they are refilled with points at every depth instead of thinning out. The run ends with 17 leaves at depths up to 6, after 1053 evaluations of f.

Algorithm 6: ART on one batch of nodes: `ARTPlan.process_batch` in `src/art/plan.py`. The notes in code type are the leaf's recorded `terminal_reason`.

Require: a batch of nodes C: region Ω_C, inherited points, path code π; budget n
Ensure: leaves $(\pi, (w, b))$; split records (π, a, z); two children per split node
1 **parallel for** each node C
2 draw new points in Ω_C until C has n; evaluate f at the new ones // *Algorithm 7*
3 $(w, b) \leftarrow$ affine fit; $e_a, e_r \leftarrow$ its errors // *as AFINNI*
4 each node within tolerance becomes a leaf // `accepted`
5 each other node at the depth limit becomes a leaf anyway // `max_depth`
6 **parallel for** each other node C
7 $(a, z) \leftarrow$ split search on the points of C // *Algorithm 8*
8 **if** a side would hold fewer than p points: C becomes a leaf // `split_failed`
9 $C_{-1} \leftarrow \{x \in C : a^\top u(x) < z\}$, $C_1 \leftarrow \{x \in C : a^\top u(x) \geq z\}$
10 their regions: Ω_C with the row $a^\top u \leq z$, or $a^\top u \geq z$, added
11 their path codes: 2π and $2\pi + 1$; record (π, a, z)

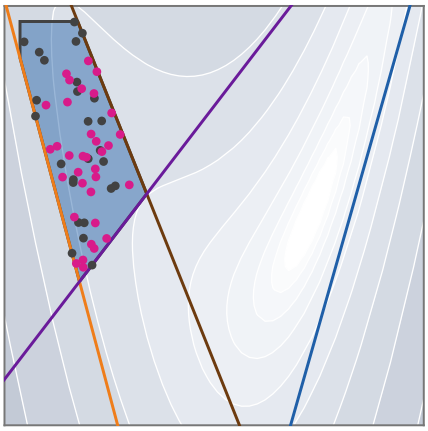
6.1 A fixed budget of points in every node

A node is fitted on $n = \lceil mp \rceil$ points, where $p = d + 1$ is the number of coefficients of an affine model and m is the `sample_multiplier`: here $m = 20$ and $n = 60$. It inherits the points of its parent that fall on its side, which cost nothing, and draws the rest uniformly in its own region. A node deep in the tree is thus fitted and tested on as many points as the root, concentrated in its small region.

1 A region is the box and its ancestors' planes

Algorithm 6

Require: a batch of nodes C: region Ω_C, inherited points, path code π; budget n
Ensure: leaves $(\pi, (w, b))$; split records (π, a, z); two children per split node
⋮
2 draw new points in Ω_C until C has n; evaluate f at the new ones
⋮

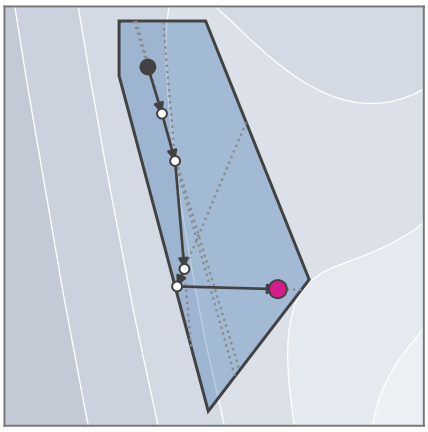


The node with path code 11111, at depth 4: its region (shaded) is bounded by the box and by the 4 planes of its ancestors (lines), one row of (A, z) each. It inherited 26 points (dark) and needs 34 new ones (pink), anywhere in the region, including next to its boundary.

2 Hit-and-run: a random walk that stays inside

Algorithm 7

Require: the region $\Omega_C = \{u : Au \le z\}$, a point $u \in \Omega_C$, a number of steps τ
Ensure: a new point of Ω_C
1 **repeat** τ times
2 $g \leftarrow$ a direction, uniform on the unit sphere
3 $[t_{\min}, t_{\max}] \leftarrow \{t : A(u + tg) \le z\}$
4 $u \leftarrow u + tg$ with t uniform in $[t_{\min}, t_{\max}]$
5 **return** u



Drawing uniformly in a polytope is done by walking. From a point inside (dark), choose a random direction; the region cuts the line through the point in a chord (dotted); jump to a uniform point of the chord. Each step only needs the two ends of the chord, which come from one product Ag. After a few steps the walker (pink) has forgotten where it started.

Algorithm 7: Hit-and-run in a polytope [5], [6]: `HitAndRunSampler` in `src/art/sampling.py`. One chain per new point, started at a random inherited point.

Require: the region $\Omega_C = \{u : Au \le z\}$, a point $u \in \Omega_C$, a number of steps τ
Ensure: a new point of Ω_C
1 **repeat** τ times
2 $g \leftarrow$ a direction, uniform on the unit sphere
3 $[t_{\min}, t_{\max}] \leftarrow \{t : A(u + tg) \le z\}$ *// the chord through u*
4 $u \leftarrow u + tg$ with t uniform in $[t_{\min}, t_{\max}]$
5 **return** u

Proposition 1 (hit-and-run samples uniformly, [5]). The uniform distribution on a bounded convex region with interior is the unique stationary distribution of the hit-and-run walk, and the walk converges to it from any interior point.

Finitely many steps. Proposition 1 is a statement about the limit. A chain of τ steps in a long thin region may not have crossed it yet, and its points then cluster near their starts. The code therefore *checks*: it compares the points of each node halfway through the walk and at its end, and if their mean has drifted by more than `mixing_z` standard errors, it continues those chains, doubling τ from `walk_steps` up to `max_walk_steps`. A node that still drifts is fitted anyway and, if it becomes a leaf, is reported with the `validation_status` `insufficient_sampling`, so a model with unmixed regions cannot pass silently.

Invariant 2 (random numbers belong to nodes). Every random number ART draws is a hash of the run's seed, the node's root and path code, the purpose of the draw and a counter (`src/art/rng.py`). No generator is advanced from batch to batch. The points a node draws, and so its fit and its plane, are therefore the same whatever batch the node is in, whichever rank processes it, and whenever it is stolen (Section 10).

In the code.

- `roots="grid"` or `"voronoi"` starts from several roots instead of one: a regular grid of the domain, or the Voronoi cells of the initial clustering's centres. More roots give the ranks independent work from the first level on.
- A root box is sampled exactly, without a walk.
- `validation_samples` draws a second, independent set of points in every node, on which an accepted model is checked again before it becomes a leaf.

- On a GPU the whole walk of every chain of a batch runs in one kernel (`src/kernels/hit_and_run.cu`), one thread per chain.

In one line. An ART node owns a polytope, the box cut by its ancestors' planes, and is fitted on a fixed number of points drawn uniformly inside it by hit-and-run.

7 ART: choosing the plane

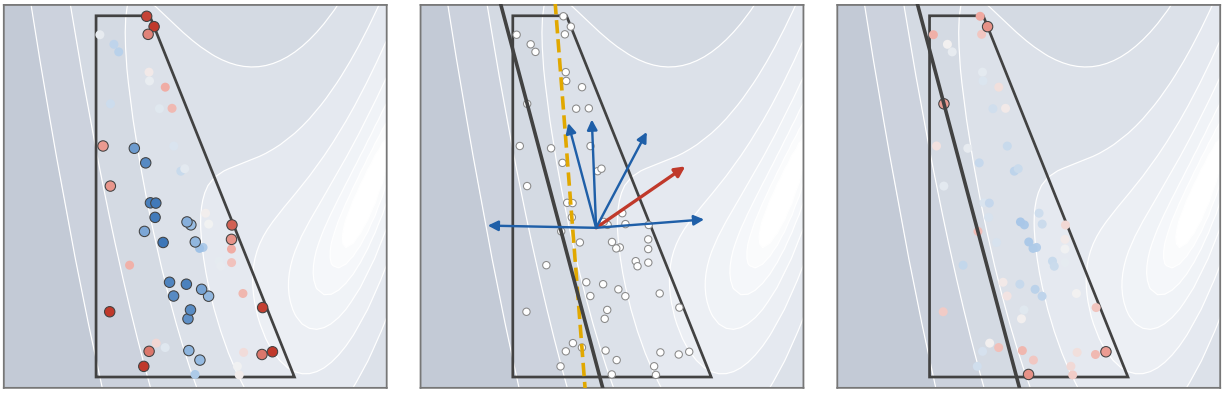
A node is rejected. Of all the planes that cut its region in two, which one?

AFINNI computes its cut from the curvature in closed form. ART *searches* for it: the best plane is the one after which two affine models, one per side, fit the node's points best, which is what the children's least-squares fits will minimize.

Definition 3 (the split objective). For a node with points u_i and values $y_i = f(x_i)/\alpha_{\text{tol}}$, the *hard* objective of a plane (a, z) is the squared error of the two children's least-squares fits,

$$\text{SSE}(a, z) = \min_{\varphi_{-1},\varphi_1 \text{ affine}} \sum_i \begin{cases} (y_i - \varphi_{-1}(u_i))^2 & \text{if } a^\top u_i < z \\ (y_i - \varphi_1(u_i))^2 & \text{if } a^\top u_i \geq z \end{cases} \tag{3}$$

among planes that leave at least p points on each side.



a rejected node: the error of its one model

candidate directions; the best cut found (dashed), then refined (solid)

the errors of the two children's models

Figure 4: The split search on the node with path code 111, with 60 points. One model leaves a mean squared error of 2.21 (in units of α_{tol}^2). The search tries cuts along 6 directions (arrows; the red one is AFINNI's Hessian-informed direction) and finds a plane with error 0.508 the soft search turns and shifts it to reach 0.32.

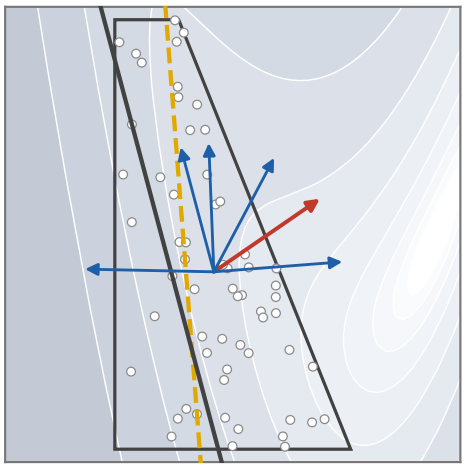
7.1 A coarse search, then a smooth one

The hard objective is a staircase: it changes only when the plane crosses a point, and has no useful gradient. ART handles it in two stages.

1 Many cuts along a few directions

Algorithm 8

directions ← the Hessian-informed one, further eigenvectors, the points' principal one, random ones
for each direction a and each of $n_{\text{bins}} - 1$ cuts z at quantiles of $a^\top u$
    score (a, z) by the squared error of two affine fits, one per side
(a, z) ← the best candidate

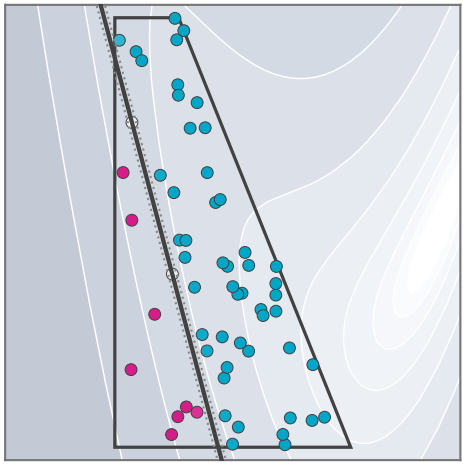


First a search by enumeration. Along each of $n_{\text{dir}} = 6$ directions the node's points are sorted, and $n_{\text{bins}} - 1 = 15$ cuts are tried, at equal fractions of the points. Every candidate's two fits come from running sums over the bins, so a direction costs one pass over the points. The best candidates are refitted exactly; the winner is the dashed line.

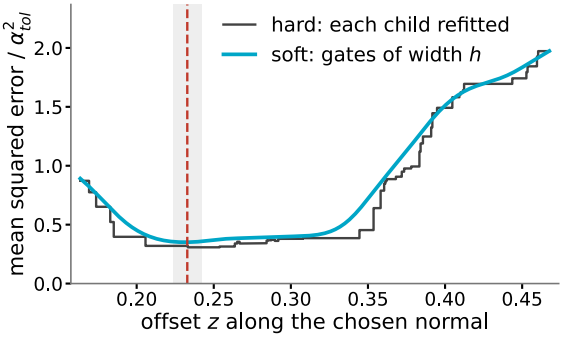
2 Soft gates make the objective smooth

Algorithm 8

repeat at most `soft_iters` times
    step (a, z) down the gradient of J_h, keeping $\|a\| = 1$



Then a search by descent. Replace "left or right" by a *gate*: a point at signed distance $a^\top u - z$ from the plane belongs to the right child with weight $\sigma((a^\top u - z)/h)$, where σ is the logistic function and h the gate width (the dotted band). Far from the plane the points are pink or cyan as before; in the band they are shared. The two models become weighted fits, and their error a smooth function of (a, z).



Both objectives along the chosen normal, as functions of the offset z. The hard one (dark) is a staircase with a step at every point. The soft one (cyan) follows it and can be differentiated: an optimizer can slide down it, turning the normal as well as moving the offset, which the enumeration could not do. The red line is the plane that was chosen and the band is the gate width around it.

Definition 4 (the soft objective). With weights $W_1(s) = \delta + (1 - 2\delta)\sigma(s)$ and $W_{-1}(s) = \delta + (1 - 2\delta)\sigma(-s)$, $s_i = (a^\top u_i - z)/h$, a small floor δ and a fixed ridge λ,

$$J_h(a, z) = \min_{B_{-1}, B_1} \frac{1}{n} \left[\sum_i \sum_{k=\pm 1} W_k(s_i) \big(y_i - B_k^\top \varphi_i\big)^2 + \lambda \big(\|B_{-1}\|^2 + \|B_1\|^2 \big) \right], \tag{4}$$

where $\varphi_i = (u_i, 1)$ and B_k are the coefficients of the two models.

The minimization over the models is a pair of weighted least-squares problems with a closed-form solution, so only the plane is optimized: this is *variable projection* [7]. Its gradient needs no derivative of the models.

Lemma 4 (the gradient of the soft objective). Let $r_{k,i} = \left(y_i - B_k^\top \varphi_i\right)^2$ at the minimizing models. Then

$$\nabla_a J_h = \frac{1}{nh} \sum_i \left(r_{1,i} - r_{-1,i}\right)(1 - 2\delta)\sigma'(s_i)u_i, \quad \partial_z J_h = -\frac{1}{nh} \sum_i \left(r_{1,i} - r_{-1,i}\right)(1 - 2\delta)\sigma'(s_i).$$

Proof. J_h is a minimum over B of a function that is differentiable in (a, z), and the minimizer is unique because of the ridge. By the envelope theorem the derivative of the minimum is the partial derivative at the minimizer: only the weights depend on (a, z). □

The sum has a clear meaning: a point in the band pulls the plane towards the side whose model fits it worse. Each step moves along the negative gradient, projected on the tangent plane of the sphere $\|a\| = 1$ and followed by renormalizing a, with a backtracking line search that keeps the models frozen.

Lemma 5 (frozen models still give descent). Let $F(\theta, B)$ be the function minimized over B in Eq. 4, at plane $\theta = (a, z)$ and models B, and B_t the minimizing models at the current plane θ_t. If a trial plane θ' satisfies $F(\theta', B_t) < J_h(\theta_t)$, then $J_h(\theta') < J_h(\theta_t)$.

Proof. $J_h(\theta') = \min_B F(\theta', B) \le F(\theta', B_t) < J_h(\theta_t)$. □

So a trial step is judged with the current models, which costs one pass over the points, and the models are refitted only after a step is accepted. This holds because the same weights, floor and ridge appear in the fit and in the objective.

Invariant 3 (the stored plane defines the children). After the search, z is moved to the score of the first point on the right, and the children are *defined* by the stored predicate $a^\top u \ge z$. The code verifies that the predicate reproduces the cut, point by point, and raises an error otherwise. Training and routing use the same function `plane_scores` to evaluate it, so a point can never be on one side when it is fitted and on the other when it is routed.

In the code.

- The soft result is kept only if its two *hard* fits are better than the enumeration's. The soft search can only improve a split: on the example it did for 7 of the 16 nodes that were split.
- h is `soft_temperature` $= 0.1$ times the standard deviation of the scores $a^\top u$, and at most `soft_iters` $= 40$ steps are taken.
- `split_search="bins"` stops after the enumeration, and `"gap"` cuts along the Hessian-informed direction near the median without any search.
- The values are divided by the tolerance of their component first (the larger of α_{tol} and r_{tol} times the node's largest $|f|$), so that for vector-valued f a component is weighted by how far it is from *its* tolerance.

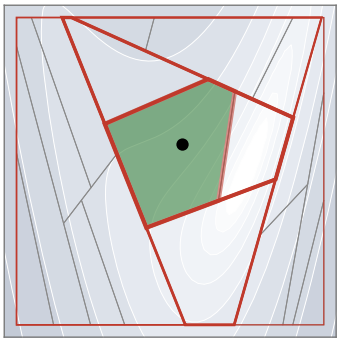
Algorithm 8: ART's split search: `_bins_search` and `_soft_refine` in `src/art/plan.py`, on the primitives of `src/art/split_search.py`.

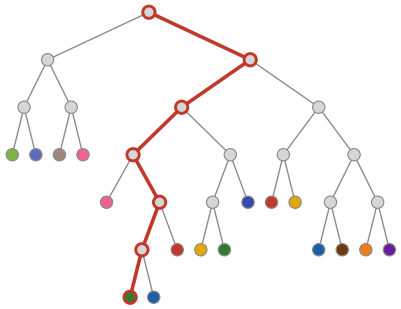
Require: the points of a node in normalized coordinates u, their values in units of the tolerance
Ensure: a plane (a, z) with $\|a\| = 1$

directions ← the Hessian-informed one, further eigenvectors, the points' principal one, random ones // n_{dir} *in all*
for each direction a and each of $n_{\text{bins}} - 1$ cuts z at quantiles of $a^\top u$
    score (a, z) by the squared error of two affine fits, one per side
(a, z) ← the best candidate // *the best few are refitted exactly*
repeat at most `soft_iters` times // *soft search*
    step (a, z) down the gradient of J_h, keeping $\|a\| = 1$ // *Eq. 4*
move z to the first point on the right; keep the result only if its two hard fits are better

7.2 From a tree to an answer

A query is *routed*: start at the root, and at every split node evaluate $a^\top u - z$ and go right if it is non-negative. The picture follows one query (black dot) through the nested regions it falls in, down to its leaf at depth 6 the path code of the leaf, 1100100, is the record of the turns taken. The leaf's model gives the answer.





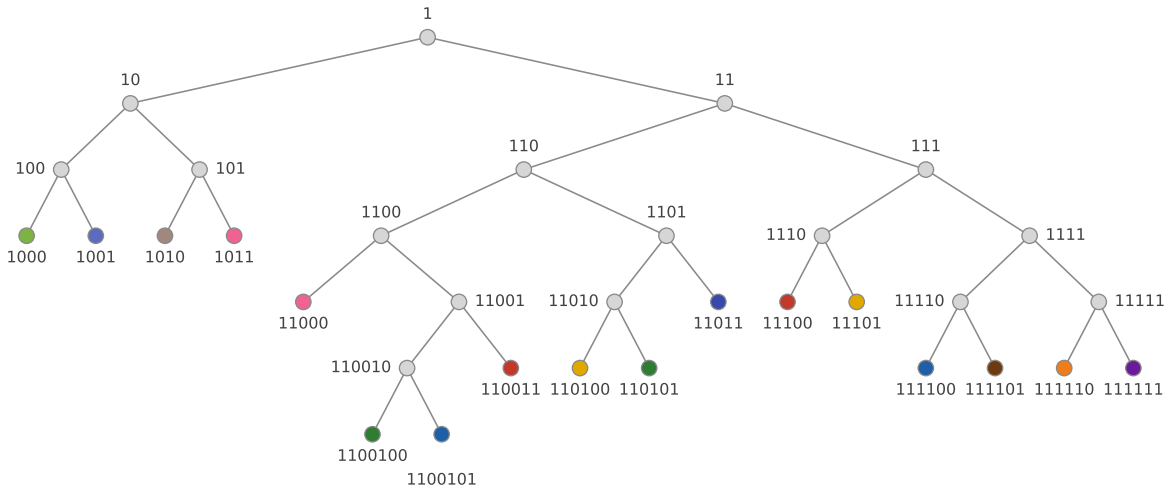


Figure 5: The tree of the running example with its path codes in binary. A node's code is its parent's with one digit appended, 0 for the side $a^\top u < z$ and 1 for the side $a^\top u \geq z$; the number of digits after the leading 1 is the depth.

Theorem 3 (the leaves partition the domain). Every point of the domain is routed to exactly one leaf, and it lies in that leaf's region. The cost is one inner product per level: $O(d \cdot \text{depth})$.

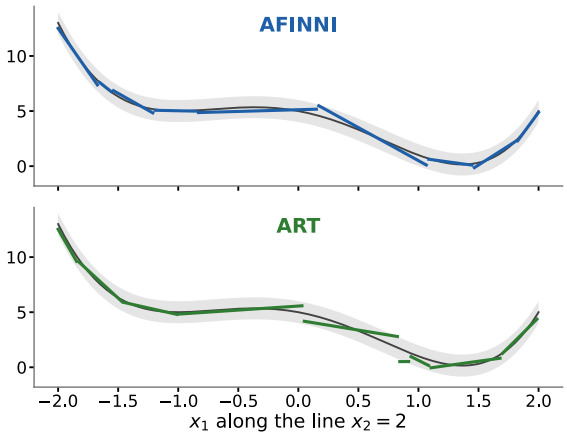
Proof. At each split node the predicate $a^\top u \geq z$ is either true or false, so exactly one child is entered, and by Definition 2 the point lies in that child's region if it lay in the parent's. The root's region is the whole domain. □

In the code. The `RoutingTable` (`src/art/routing.py`) stores the tree as arrays: for every node its two children, its plane and, for a leaf, its row in the leaf arrays. It is assembled on rank 0 after training, from the split records (π, a, z) of all ranks, and *checked*: every split has both children, every node but a root has a parent, no code appears twice. A query outside the domain is routed from its nearest point of the domain and returned flagged, as AFINNI flags a query outside its trust regions.

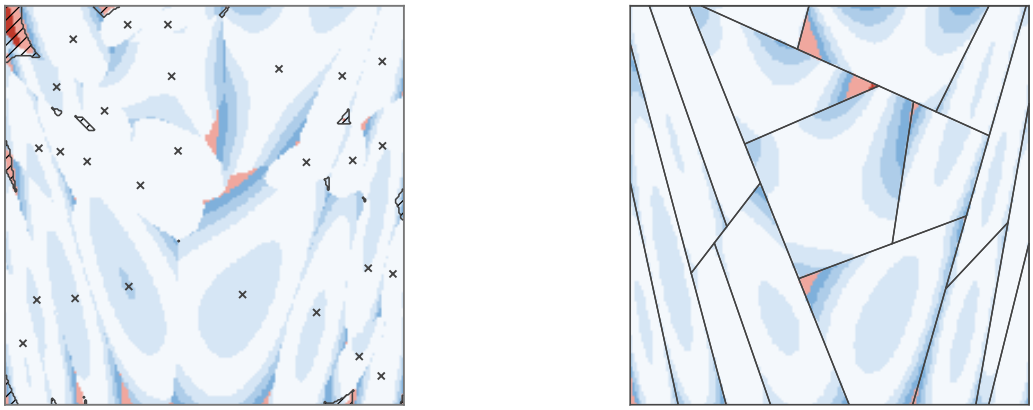
In one line. ART picks the plane whose two children fit best: an enumeration over cuts along a few directions, then gradient descent on a smooth version of the same objective, kept only if it helps. The planes are stored, so the leaves tile the domain and a query walks to its leaf.

8 Side by side

The same function, the same tolerance, two algorithms. What does each one give?



First along one line across the valley, $x_2 = 2$. The dark curve is f, the grey band is $f \pm \alpha_{\text{tol}}$, and each coloured segment is one leaf's affine model. Both surrogates are piecewise affine and stay inside the band almost everywhere; both have small jumps where one leaf hands over to the next. The pieces are short where f curves and long where it is nearly straight: that is the adaptivity.



AFINNI: 27 leaves ART: 17 leaves

Figure 6: The error $\left|f - \hat{f}\right|/\alpha_{\text{tol}}$ over the whole domain, on a 240 × 240 grid: pale to blue within the tolerance, to red beyond it. Left, AFINNI's centres (crosses) and, hatched, the queries flagged as outside every trust region. Right, ART's leaves.

On the running example, $\alpha_{\text{tol}} = 1$	**AFINNI**	**ART**
leaves	27	17
evaluations of f	1189	1053
of which at new points, besides the centres	539	960
share of the domain within the tolerance	98.4%	99.1%
share of the domain flagged	1.5%	0%
largest error on the grid, in units of α_{tol}	3.44	2.35

Neither surrogate is within the tolerance everywhere, and neither claims to be: the tolerance was enforced at training points, and between them a model can exceed it, mostly near the edges of leaves. What differs is what each algorithm can say about it.

AFINNI

Works from the sample it is given, wherever that lies, and evaluates f beyond it only inside the hulls of its clusters. Its flag marks the queries no leaf has data for; here they are at the rim of the box, beyond the outermost points. Inside the trust regions 99.1% of the grid is within the tolerance.

ART

Covers the whole domain by construction and flags nothing inside it. Because its leaves partition a known domain, an independent check is possible: `audit` draws fresh uniform points and bounds the share of the domain that violates the tolerance.

Proposition 2 (the audit bound, [8]). Let k of n independent uniform points of the domain violate the tolerance. With confidence η, the share of the domain that violates it is at most the Clopper–Pearson bound, the η-quantile of the $\mathrm{Beta}(k+1, n-k)$ distribution; with $k = 0$ this is $1 - (1-\eta)^{1/n}$.

On the example, `audit` with 2000 fresh points finds 24 violations (1.2%), so with 95% confidence at most 1.7% of the domain violates the tolerance. The bound holds for the trained model and the uniform measure; it does not bound the error at any particular point.

Which one. The two are not competitors so much as answers to different situations. Use AFINNI when the data come first: a large sample from a simulation, no natural box around it, and the need to know when a query leaves the region the data cover. Use ART when the domain comes first: a box that must be covered, an oracle that can be called anywhere in it, and the wish for a tree, exact routing and an audit. `examples/example05_afinni_vs_art.py` runs both on the same problems.

In one line. Both meet the tolerance on almost all of the domain with a few dozen affine pieces; AFINNI flags where it has no data, ART covers everything and can be audited.

9 Batches, memory and reproducible sums

> *The engine handles thousands of clusters at once on a GPU. How, without a loop over clusters, and without results that change from run to run?*

By keeping the layout of Figure 1 all the way down. A batch is three arrays, the points `X`, the values `F`, and the cluster sizes, with every cluster a run of consecutive rows. Every step of Algorithm 2 and Algorithm 6 is then one of two kinds of array operation.

Segmented reductions
A sum, maximum or scatter matrix over the rows of each cluster: centroids, Σ, the errors e_a. One pass over the rows produces one result per cluster.

Small batched solves
A $d \times d$ or $p \times p$ problem per cluster: the least-squares fit, an eigendecomposition for the cut or the trust ellipsoid. All clusters are solved together.

Constraint	What the engine does
the points of a level can exceed the device's memory	keeps the queue in host memory and moves one batch at a time to the device
a batch holds as many whole clusters as fit	caps `batch_size` (points) at 65% of the free device memory divided by `plan.bytes_per_point()`
a batch's points must be released as early as possible	hands them to the plan (`DeviceBatch.take`), which frees them before it augments and cuts
the host can run out of memory too	checks the available memory before the queue grows, and raises `MemoryError` with advice

9.1 The same bits every time

Floating-point addition is not associative. A sum computed by many GPU threads in whatever order they finish differs from run to run in its last bits, and a last bit can decide whether a cluster passes its tolerance, and so the shape of the whole tree.

Invariant 4 (deterministic sums). Every sum on the training path is taken in a fixed order that depends only on the cluster it belongs to. On the GPU, a cluster's rows are summed sequentially in chunks counted from its own first row, and the chunks are then added in order (`segment_cross`, `reduce_sum_1d`,

`det_matmul`, `spd_solve` in `src/numerics/`). No thread adds into a shared total, and no library routine chooses its algorithm by the size of the batch.

Proposition 3 (what does not change the result).
1. Repeated runs in one process on the same backend give bit-identical models.
2. An ART tree, as the set of its planes by path code, does not depend on the batch size, the number of ranks or the schedule of steals.
3. An AFINNI forest is fixed by the seed, the batch size and the number of ranks, when work stealing is off.

Proof. (1) follows from Invariant 4 and the seeded random streams. (2) A node's points depend only on its key (Invariant 2), its sums only on its own rows (Invariant 4), and its plane only on those; by induction from the roots, so does every node. (3) AFINNI's new points are drawn from a generator seeded per batch, so they depend on which clusters share a batch; that is fixed by the batch size and the scatter over ranks, but a steal moves clusters between batches. □

CPU and GPU. A CPU run and a GPU run agree to rounding, about 10^{-12}, not bit for bit: their sums are taken in different orders. The tests compare them with a tolerance that tight, and require leaf counts and evaluation counts to match exactly.

In one line. Clusters are runs of rows, batches are runs of clusters, and every sum is taken in an order that only its own cluster determines: fast on a GPU, and the same on every run.

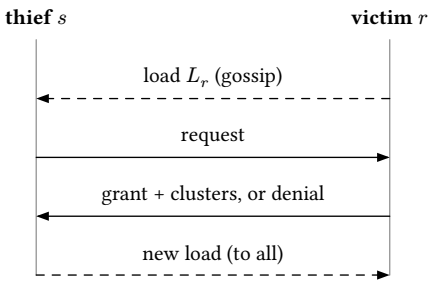
10 Many ranks

> *Refinement is uneven: one region of the domain may need ten times the levels of another. How do many processes share work that nobody can predict?*

Rank 0 evaluates f on the initial points, builds the initial queue and cuts it into P contiguous runs with equal numbers of clusters, one per rank. From then on every rank runs Algorithm 1 on its own queue. A static division cannot stay balanced, because the work a cluster will generate is not known until it has been fitted: a rank whose clusters are accepted early runs dry while another is many levels deep.

So idle ranks *steal* [9]. The roles are a thief, with an empty queue, and a victim, with work to spare.

Each rank publishes its load, the points waiting in its queue, to all others at every level boundary and after every steal: an 8-byte message, never waited for. A thief asks the rank it believes is the busiest. The victim looks at requests only *between batches*, so it is never interrupted in the middle of one, and answers every request with a grant or a denial. A grant is followed by the clusters themselves: whole clusters from the *tail* of the victim's queue, the ones it would have reached last.



Algorithm 9: Decentralized work stealing, run by every rank i (the manuscript's Algorithm 7): `WorkStealer` in `src/engine/work_stealing.py`.

Require: rank i of P; its work w_i (points); threshold w_*; steal size w_s
Ensure: every cluster is processed by exactly one rank

while not retired
  $L \leftarrow$ loads the other ranks have published *// gossip, never blocks*
  if $w_i > 0$ *// **victim***
    process one batch
    for each steal request from a rank s
      if more than w_* of unstarted work is left: send s clusters from the tail of the queue
      else send s a denial *// always one reply*
  else *// **thief***
    publish $w_i = 0$; deny every request
    if $\sum_j L_j = 0$: join the barrier; retire once every rank has joined
    $r \leftarrow \arg\max_j L_j$; **if** $L_r \geq w_*$: ask r and wait for its grant or denial

Invariant 5 (the stealing protocol).

1. **One reply.** Every request receives exactly one grant or denial. A thief waits for that explicit reply and for nothing else, so a stale view of the loads can delay it but not deadlock it.
2. **Only unstarted work.** A grant takes whole clusters from behind the current batch and always leaves the victim at least one. A cluster is processed by exactly one rank.
3. **A cluster travels complete.** A stolen cluster carries its points, values, depth and, for ART, its root, path code and the rows of its region. The thief needs nothing else from the victim, then or later.
4. **Nobody leaves early.** A rank retires only when every rank is idle: it joins a non-blocking barrier and keeps answering requests until all have joined.

10.1 One recorded run

Stealing depends on timing, so its picture is of one recorded run: ART on the running example with a tighter tolerance, $\alpha_{\text{tol}} = 0.05$, on 4 ranks. ART starts from a single root, so rank 0 holds all the work at first and every other rank must steal.

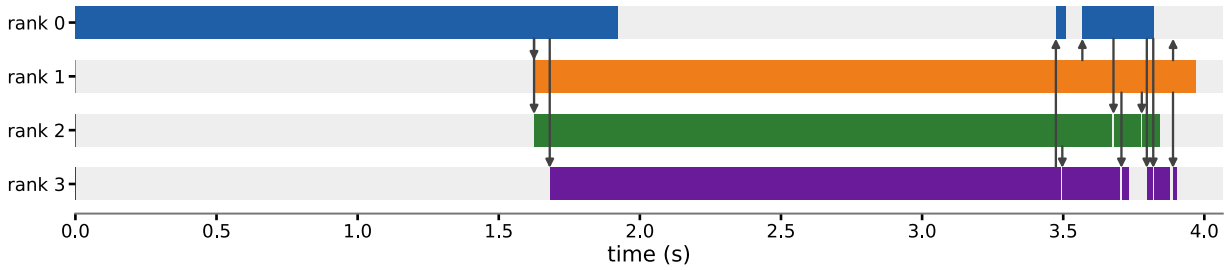
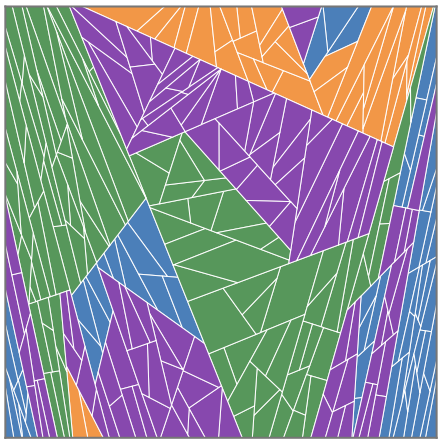


Figure 7: Who was working when. A rank's bar is coloured while it has nodes and pale while it is idle; each arrow is a grant, from the victim to the thief. Rank 0 builds the first levels alone (including the one-time compilation of the CPU kernels); once its queue is long enough to share, the other ranks take work from it and, later, from each other: 13 grants in all.



The same run as a map: each of the 283 leaves, coloured by the rank that built it (56, 32, 93, 102 leaves per rank). A stolen node takes its whole subtree with it, so each rank's leaves form patches of the domain.

The tree itself is the one a single process builds. By Proposition 3 every node's points and plane depend only on its key, so it does not matter which rank processed it; the recording script checks that every plane of this tree, by path code, is bit-identical to the tree built with stealing switched off.

In the code.

- The threshold w_* and the steal size w_s are counted in points: AFINNI grants when more than 10^6 unstarted points remain, ART when a few nodes' worth do (`steal_min_load`, `steal_size`). A grant gives about $\max(w_s, \text{half the unstarted work})$.
- ART's split search can be long, so it also answers requests *inside* a batch, through `batch.progress()`, still giving away only clusters behind the batch.
- Under MPI an exception on one rank would leave the others waiting in a collective call. `run_collective` therefore logs the rank, phase and traceback and aborts every rank; in a single process the exception simply propagates.
- `python -m src.infra.monitor` shows each rank's level, queue and steals live, from files the ranks write, without any MPI traffic.

In one line. Ranks start with equal shares and stay busy by stealing unstarted clusters from the tail of the busiest rank's queue; every request gets one answer, a cluster travels complete, and for ART the tree does not depend on who built it.

11 How we know it is right

> *Thousands of batched operations, two backends, many ranks. What stands between a change to the code and a wrong surrogate?*

Definitions
Each component is tested against its formula on small cases: the fit recovers an affine function exactly, the errors match Eq. 2, the trust ellipsoid against Lemma 3, a plane against the cut it must reproduce.

Both plans, one contract
`test_plans.py` holds AFINNI and ART to the `Plan` protocol, checks their batch boundaries, and checks that an AFINNI batch's points are released before splitting.

Differential regression
`scripts/regression_check.py` trains 52 small cases covering every mode, the `L2` and `Linf` metrics and every split search, and stores every array of every model. A refactor must reproduce them bit for bit on a CPU, on one rank and on two.

CPU against GPU
The GPU suite runs the same tests and compares with a tolerance of 10^{-12} (10^{-10} between two separate trainings); leaf counts and evaluation counts must match exactly.

Many ranks
`tests/mpi` runs real `mpiexec` jobs: scatter and gather, stealing with gradients, ART's tree against a single-rank run, and that a failure on one rank aborts all.

Structure
`test_layering.py` enforces the import layers; `test_kernel_sources.py` checks, on any machine, that every CUDA kernel has a caller and every call a kernel.

Three checks are built into the library itself, because they concern a particular model more than the code. The routing table is validated when it is assembled and again when a model is loaded (Section 7). ART reports every leaf's `terminal_reason` and `validation_status`, so a tree with fallback leaves or unmixed samples says so. And `audit` measures a trained ART model on fresh points (Proposition 2).

This companion's figures are a test of the same kind: the script that draws them rebuilds AFINNI's forest from the recorded queue, ART's polytopes from the stored planes, the affine fits, the Hessian estimate and the inference rule from their formulas, and stops if any of them disagrees with the library.

In one line. Formulas are tested on small cases, whole models are compared bit for bit across refactors, backends and ranks, and a trained model carries its own diagnostics.

12 Using it

```
from src.config import ARTSettings, TrainingConfig
from src.core.evaluation import Evaluator
from src.surrogate import AFINNISurrogate

evaluator = Evaluator(f_fn=f)                    # f maps (N, d) points to (N, fd) values

afinni = AFINNISurrogate()                       # AFINNI: from a sample X0
afinni.train(X0, TrainingConfig(atol=1.0, num_initial_clusters=4, min_pts_per_cluster=24), evaluator)
values, outside = afinni.predict(queries, k_nn=1, coarse_factor=10)

art = AFINNISurrogate()                          # ART: on a domain
art.train(X0, TrainingConfig(atol=1.0, algorithm="art", domain=(lower, upper),
                             art=ARTSettings(sample_multiplier=20)), evaluator)
values, outside = art.predict(queries)           # routed down the tree
report = art.audit(evaluator, n_points=2000)     # share of the domain beyond the tolerance
```

These are the main settings of the running example (it also fixes `seed` and `max_levels`). The backend is chosen by the environment: `AFINNI_DEVICE=cpu|gpu|auto` and `AFINNI_MPI=off|on|auto`; under `mpiexec`, `train` and `predict` are collective. `docs/USAGE.md` lists every setting, `docs/ARCHITECTURE.md` maps the stages above to the modules of `src/`, and `docs/AFINNI_ART_DESIGN.md` records the design decisions behind them.

References

[1] "AFINNI: Adaptive Affine Nearest Neighbor Interpolation in High Dimensions," 2026.

[2] *Adaptive Regression Trees (ART): reference implementation.* (2026).

[3] C. Ding and X. He, "K-means clustering via principal component analysis," in *Proceedings of the 21st International Conference on Machine Learning (ICML)*, 2004.

[4] M. J. Todd, *Minimum-Volume Ellipsoids*. Society for Industrial and Applied Mathematics, 2016.

[5] R. L. Smith, "Efficient Monte Carlo procedures for generating points uniformly distributed over bounded regions," *Operations Research*, vol. 32, no. 6, pp. 1296–1308, 1984.

[6] L. Lovász and S. Vempala, "Hit-and-run is fast and fun," *Microsoft Research technical report MSR-TR-2003-05*, 2003.

[7] G. H. Golub and V. Pereyra, "The differentiation of pseudo-inverses and nonlinear least squares problems whose variables separate," *SIAM Journal on Numerical Analysis*, vol. 10, no. 2, pp. 413–432, 1973.

[8] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[9] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," *Journal of the ACM*, vol. 46, no. 5, pp. 720–748, 1999.